

**ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT
MÁY TÍNH**



Báo cáo Bài Tập Lớn

Game sắp xếp block

GVHD: TS. Trương Vĩnh Lân

SVTH 1: Phạm Ngọc Long	2211894
SVTH 2: Nguyễn Ngọc Duy	2210522
SVTH 3: Võ Quốc Phong	2352913
SVTH 4: Nguyễn Tấn Đạt	2352234

TP. HỒ CHÍ MINH, 2026

Mục lục

1	Giới thiệu	2
1.1	Bối cảnh bài toán	2
1.2	Phát biểu bài toán	2
1.3	Thông tin tra cứu	3
2	Biểu diễn & tìm kiếm	4
2.1	Mô hình hóa bài toán	4
2.2	Thuật toán tìm kiếm sử dụng	7
2.2.1	Best-first Search cho một lượt	7
3	Thiết kế heuristic cho hệ thống tìm kiếm	10
3.1	Vai trò của heuristic trong hệ thống	10
3.2	Hàm heuristic được sử dụng	10
3.3	Đánh giá thực nghiệm các trọng số (Ablation Study)	12
3.3.1	Kịch bản Baseline	12
3.3.2	Kịch bản 1: Loại bỏ yếu tố kiểm soát phân mảnh($w_1 = 0$)	12
3.3.3	Kịch bản 2: Loại bỏ yếu tố kiểm soát diện tích bao phủ($w_2 = 0$)	12
3.3.4	Kịch bản 3: Tối đa hóa cục bộ việc xóa hàng ($w_3 \gg w_1, w_2$)	13
4	Biểu diễn và suy luận tri thức	14
4.1	Tri thức bổ sung trong hệ thống	14
4.2	Cơ chế tie-break trong hệ thống	15
5	Mạng Bayes và suy diễn xác suất rủi ro	16
5.1	Mục tiêu của chương	16
5.2	Cơ sở lý thuyết	16
5.3	Định nghĩa các biến ngẫu nhiên	17
5.4	Phạm vi thông tin	18
5.5	Cấu trúc mạng Bayes	18
5.6	Bảng xác suất có điều kiện CPD	19

5.7	Suy diễn xác suất	19
5.8	Kết quả thực nghiệm của module Bayes	20
5.9	Vai trò của module Bayes trong hệ thống	21
5.10	Độ phủ dữ liệu và hướng mở rộng	21
6	Học máy	23
6.1	Vai trò của học máy trong hệ thống	23
6.2	Mô hình hóa bài toán học máy	24
6.3	Sinh dữ liệu huấn luyện	24
6.4	Trích xuất đặc trưng	24
6.5	Thuật toán Decision Tree	26
6.6	Huấn luyện và đánh giá mô hình	26
6.7	Ý nghĩa và nhận xét về mô hình học máy	27
7	Kết luận	i
A	Phân chia công việc nhóm	ii
B	Lịch sử sửa đổi	iii
C	Lịch sử cuộc họp	iv

Chương 1

Giới thiệu

Trong bài tập lớn này, nhóm lựa chọn xây dựng một bài toán xếp khối trên bàn cờ dưới góc nhìn của trí tuệ nhân tạo. Bài toán có thể được mô hình hóa rõ ràng bằng các thành phần cơ bản như trạng thái, hành động và mục tiêu. Từ đó, hệ thống có thể áp dụng các phương pháp tìm kiếm để lựa chọn cách đặt khối phù hợp nhằm đạt được kết quả tốt nhất.

1.1 Bối cảnh bài toán

Bài toán xếp các khối hình học vào một không gian có giới hạn là một dạng bài toán quen thuộc trong trí tuệ nhân tạo và tối ưu hóa. Trong những bài toán này, mỗi quyết định đặt khối ở hiện tại đều ảnh hưởng trực tiếp đến khả năng sắp xếp các khối tiếp theo, vì vậy cần có một chiến lược hợp lý thay vì chỉ chọn ngẫu nhiên.

Trong đề tài này, nhóm xét bài toán trên một bàn cờ kích thước 8×8 . Quá trình chơi được chia thành nhiều lượt, và ở mỗi lượt hệ thống cung cấp 4 khối hình học để đặt lên bàn cờ sao cho hợp lệ. Chỉ khi đặt hết 4 khối của lượt hiện tại thì hệ thống mới cung cấp 4 khối tiếp theo. Sau mỗi lần đặt, nếu một hàng hoặc một cột được lấp đầy hoàn toàn thì hàng hoặc cột đó sẽ được xóa khỏi bàn cờ, đồng thời người chơi nhận được điểm số tương ứng. Vì vậy, bài toán không chỉ yêu cầu đặt khối đúng mà còn cần lựa chọn cách đặt để tối đa hóa điểm đạt được trong toàn bộ số lượt chơi đã cho.

1.2 Phát biểu bài toán

Bài toán được thực hiện trên một bàn cờ kích thước 8×8 , trong đó mỗi ô có thể ở trạng thái trống hoặc đã được chiếm. Trong quá trình chơi, hệ thống tiến hành qua một số lượt xác định trước; tại mỗi lượt, hệ thống cung cấp 4 khối hình học và cần

lựa chọn cách đặt các khối này lên bàn cờ sao cho hợp lệ.

Mỗi khối chỉ được đặt tại những vị trí phù hợp với kích thước bàn cờ và không được chồng lấp lên các ô đã có trước đó. Sau khi một khối được đặt, nếu một hàng hoặc một cột trên bàn cờ được lấp đầy hoàn toàn thì các ô trên hàng hoặc cột đó sẽ bị xóa khỏi bàn cờ. Điểm số đạt được được tính dựa trên số ô biến mất sau mỗi lần xóa hàng hoặc cột. Khi toàn bộ 4 khối của một lượt đã được đặt xong, hệ thống sẽ chuyển sang lượt tiếp theo và cung cấp 4 khối mới.

Nhiệm vụ của bài toán là tìm ra cách sắp xếp các khối trên bàn cờ sao cho vừa đảm bảo tính hợp lệ, vừa đạt được tổng điểm cao nhất sau toàn bộ số lượt chơi. Đây là cơ sở để xây dựng mô hình bài toán và áp dụng các thuật toán tìm kiếm trong các phần tiếp theo.

1.3 Thông tin tra cứu

- Link GG Colab: https://colab.research.google.com/drive/1-7p8F6Q2b_-ekDuZ0juHKjkqIsp6AZJb?usp=sharing
- Link Datasets: None

Chương 2

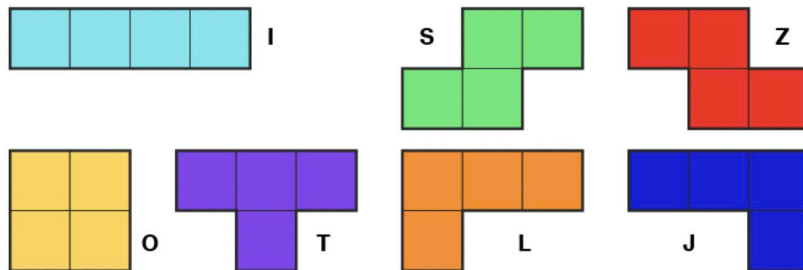
Biểu diễn & tìm kiếm

2.1 Mô hình hóa bài toán

Để đơn giản hóa, các block trong đề bài ban đầu chỉ giới hạn trong các khối Tetriminos dưới đây:

Tetriminos

Tetris shapes and their relative colors and letters



Hình 2.1: Tetriminos

Tuy nhiên, để trò chơi linh hoạt hơn, các block được phép xoay theo các góc cố định:

Listing 2.1: Tetriminos nhưng có phiên bản xoay

```
1 base = {  
2     'O': np.array([[1, 1],  
3                   [1, 1]]),  
4     'I': np.array([[1, 1, 1, 1]]),  
5     'S': np.array([[0, 1, 1],  
6                   [1, 1, 0]]),  
7     'Z': np.array([[1, 1, 0],
```

```

8         [0, 1, 1])),
9     'L': np.array([[0, 0, 1],
10                    [1, 1, 1]]),
11     'J': np.array([[1, 0, 0],
12                    [1, 1, 1]]),
13     'T': np.array([[0, 1, 0],
14                    [1, 1, 1]])
15 }
16
17 # rotate base
18 TETRIMINOS = {
19     'O': base['O'],
20
21     'I_0': base['I'],
22     'I_90': np.rot90(base['I']),
23
24     'S_0': base['S'],
25     'S_90': np.rot90(base['S']),
26     'S_180': np.rot90(base['S'], 2),
27     'S_270': np.rot90(base['S'], 3),
28
29     'Z_0': base['Z'],
30     'Z_90': np.rot90(base['Z']),
31     'Z_180': np.rot90(base['Z'], 2),
32     'Z_270': np.rot90(base['Z'], 3),
33
34     'L_0': base['L'],
35     'L_90': np.rot90(base['L']),
36     'L_180': np.rot90(base['L'], 2),
37     'L_270': np.rot90(base['L'], 3),
38
39     'J_0': base['J'],
40     'J_90': np.rot90(base['J']),
41     'J_180': np.rot90(base['J'], 2),
42     'J_270': np.rot90(base['J'], 3),
43
44     'T_0': base['T'],
45     'T_90': np.rot90(base['T']),
46     'T_180': np.rot90(base['T'], 2),
47     'T_270': np.rot90(base['T'], 3),
48 }

```

Mặc dù trò chơi tổng thể diễn ra qua nhiều lượt, trong phạm vi đề tài này nhóm

chỉ xây dựng bài toán tìm kiếm trên **một lượt chơi**. Ở đầu mỗi lượt, hệ thống nhận trạng thái hiện tại của bàn cờ cùng với 4 khối cần đặt, sau đó sử dụng thuật toán tìm kiếm để xác định thứ tự đặt và vị trí đặt phù hợp cho các khối trong lượt đó.

Bài toán được mô hình hóa như sau:

1. State (Trạng thái) - S :

Một trạng thái $s \in S$ đại diện cho cấu hình của trò chơi tại một thời điểm trong một lượt, bao gồm:

- *Board*: Ma trận nhị phân kích thước 8×8 , trong đó $Board_{i,j} = 1$ nếu ô đã có khối và $Board_{i,j} = 0$ nếu ô trống.
- *Blocks*: Danh sách các khối còn lại cần đặt trong lượt hiện tại.

2. Action (Hành động) - $A(s)$:

Hành động $a \in A(s)$ là việc chọn **một khối bất kỳ** trong danh sách $s.Blocks$ và đặt vào một vị trí hợp lệ trên bàn cờ:

- $a = (b, x, y)$ với $b \in s.Blocks$ là khối được chọn, và (x, y) là tọa độ góc trên bên trái của khối trên *Board*.
- **Điều kiện hợp lệ**: khối không nằm ngoài biên bàn cờ 8×8 và không chồng lấp lên các ô đã có giá trị 1.

Như vậy, trong mỗi trạng thái, thuật toán có thể lựa chọn cả **loại khối** lẫn **vị trí đặt**, thay vì buộc phải đặt theo thứ tự cố định.

3. Goal Test (Kiểm tra mục tiêu):

Vì bài toán tìm kiếm chỉ được xét trên một lượt, trạng thái mục tiêu là khi toàn bộ các khối của lượt đó đã được đặt xong:

$$Goal(s) = \begin{cases} True & \text{nếu } s.Blocks = \emptyset \\ False & \text{nếu ngược lại} \end{cases} \quad (2.1)$$

Ngoài ra, một trạng thái cũng được xem là kết thúc sớm nếu không còn hành động hợp lệ nào để tiếp tục đặt khối.

4. Cost (Hàm chi phí trọng số) - $C(s, a, s')$:

Để tối ưu hóa cách chơi, chi phí được định nghĩa dưới dạng hàm phạt (*Penalty*). Một nước đi dẫn đến trạng thái s' càng “xấu” thì chi phí càng cao. Trong đề tài này, chi phí được xây dựng dựa trên ba đặc trưng chính: số

thành phần liên thông còn lại, diện tích bao phủ của các khối đã đặt, và số hàng/cột được xóa. Công thức chi phí được xác định như sau:

$$Cost(s, a, s') = w_1 \cdot \Delta Holes + w_2 \cdot \Delta Area - w_3 \cdot \Delta LinesCleared + K \quad (2.2)$$

Trong đó:

- $\Delta Holes$: Độ thay đổi của số **thành phần liên thông** của không gian trống trên bàn cờ sau khi đặt khối. Giá trị này càng lớn thì không gian trống càng bị chia cắt, gây bất lợi cho các nước đi tiếp theo.
- $\Delta Area$: Độ thay đổi của diện tích **hình chữ nhật nhỏ nhất** bao phủ toàn bộ các ô đã được chiếm trên bàn cờ. Diện tích này càng lớn thì các khối càng bị phân tán và chiếm nhiều không gian hơn.
- $\Delta LinesCleared$: Số lượng hàng hoặc cột được xóa hoàn toàn sau nước đi, giúp làm giảm chi phí.
- w_1, w_2, w_3 : Các trọng số tương ứng cho từng thành phần, được xác định thông qua thực nghiệm.
- K : Hằng số đảm bảo $Cost \geq 0$, thuận tiện cho việc sử dụng trong các thuật toán tìm kiếm có chi phí.

Trong thực tế, nếu số lượng khối trong một lượt tăng lên, chẳng hạn tới 8 khối, thì việc duyệt toàn bộ không gian trạng thái theo kiểu vét cạn sẽ dẫn đến hiện tượng *bùng nổ trạng thái*. Giả sử trung bình mỗi trạng thái có khoảng b hành động hợp lệ và độ sâu tìm kiếm là d , thì số lượng trạng thái cần xét có thể lên tới $O(b^d)$. Vì vậy, nhóm lựa chọn **Best-first Search** thay cho exhaustive search để ưu tiên mở rộng các trạng thái có triển vọng hơn.

2.2 Thuật toán tìm kiếm sử dụng

2.2.1 Best-first Search cho một lượt

Best-first Search là thuật toán tìm kiếm sử dụng hàm đánh giá để xác định thứ tự mở rộng node. Thay vì duyệt toàn bộ cây trạng thái theo chiều rộng hoặc chiều sâu, thuật toán luôn ưu tiên xét trước trạng thái được đánh giá là “tốt” hơn theo một tiêu chí định trước.

Trong bài toán này, mỗi node tương ứng với một trạng thái của một lượt chơi, bao gồm cấu hình bàn cờ hiện tại và danh sách các khối còn lại cần đặt. Các node được lưu trong một hàng đợi ưu tiên (*priority queue*), trong đó node có giá trị đánh giá nhỏ hơn sẽ được mở rộng trước.

Dựa trên hàm chi phí đã xây dựng ở trên, nhóm sử dụng hàm đánh giá cho node dưới dạng:

$$Eval(n) = g(n) + \lambda \cdot |Blocks(n)| \quad (2.3)$$

trong đó:

- $g(n)$ là tổng chi phí tích lũy từ trạng thái đầu đến node n :

$$g(n) = \sum_{i=1}^k Cost(s_{i-1}, a_i, s_i) \quad (2.4)$$

- $|Blocks(n)|$ là số lượng khối còn lại chưa đặt tại node n .
- $\lambda > 0$ là hệ số điều chỉnh nhằm ưu tiên các trạng thái gần hoàn thành lượt chơi hơn.

Như vậy, node có giá trị $Eval(n)$ nhỏ hơn sẽ được xem là hứa hẹn hơn và được đưa ra mở rộng trước. Cách tiếp cận này giúp giảm số lượng trạng thái phải xét so với tìm kiếm vét cạn, đồng thời vẫn hướng thuật toán tới các cấu hình bàn cờ có chất lượng tốt.

Listing 2.2: Giả mã Best-first Search cho một lượt

```

1 BestFirstSearch(initial_state):
2   frontier <- priority queue ordered by Eval(
      node)
3   root <- Node(initial_state)
4   push root into frontier
5
6   visited <- empty map
7   best_terminal <- None
8
9   while frontier is not empty:
10    node <- pop node with smallest Eval(node)
11
12    if Terminal(node.state):
13      if best_terminal is None or
14         node.state.current_score >
           best_terminal.state.
             current_score:
15         best_terminal <- node
16      continue
17
18    key <- Key(node.state)
19
```

```

20         if key in visited and visited[key] <= Eval
           (node):
21             continue
22         visited[key] <- Eval(node)
23
24         for each action in A(node.state):
25             child_state <- Result(node.state,
                                   action)
26             child_node <- Node(
27                 state = child_state,
28                 parent = node,
29                 action = action,
30                 path_cost = g(node) + Cost(node.
                                   state, action, child_state)
31             )
32             push child_node into frontier
33
34         return best_terminal

```

Trong giả mã trên:

- *frontier* là tập các node đang chờ được mở rộng, được tổ chức dưới dạng hàng đợi ưu tiên.
- *visited* dùng để tránh mở rộng lặp lại các trạng thái đã gặp với giá trị đánh giá tốt hơn.
- *best_terminal* lưu trạng thái kết thúc tốt nhất tìm được trong suốt quá trình tìm kiếm.

Ưu điểm của Best-first Search là có thể nhanh chóng tìm được những phương án đặt khối tốt mà không cần duyệt toàn bộ cây tìm kiếm. Tuy nhiên, do thuật toán phụ thuộc mạnh vào hàm đánh giá, chất lượng nghiệm thu được cũng phụ thuộc trực tiếp vào cách lựa chọn các đặc trưng như số thành phần liên thông của vùng trống, mức độ mở rộng diện tích bao phủ và số hàng/cột được xóa.

Chương 3

Thiết kế heuristic cho hệ thống tìm kiếm

3.1 Vai trò của heuristic trong hệ thống

Trong bài toán xếp khối trên bàn cờ, số lượng trạng thái có thể phát sinh tăng nhanh theo số lượng khối cần đặt và số vị trí đặt hợp lệ của mỗi khối. Nếu duyệt toàn bộ không gian trạng thái theo kiểu vét cạn, chi phí tính toán sẽ rất lớn. Vì vậy, hệ thống cần một cơ chế đánh giá để ưu tiên mở rộng những trạng thái có triển vọng hơn.

Trong đề tài này, nhóm sử dụng thuật toán Best-first Search. Thuật toán này hoạt động dựa trên một hàm đánh giá để xác định node nào sẽ được mở rộng trước. Do đó, heuristic đóng vai trò định hướng quá trình tìm kiếm, giúp hệ thống tập trung vào những trạng thái có khả năng dẫn tới kết quả tốt hơn.

3.2 Hàm heuristic được sử dụng

Trong hệ thống đề xuất, giá trị đánh giá của một node được xác định bởi:

$$Eval(n) = g(n) + h(n) \quad (3.1)$$

Trong đó:

- $g(n)$ là tổng chi phí tích lũy từ trạng thái ban đầu đến node n ;
- $h(n)$ là giá trị heuristic ước lượng mức độ còn lại của trạng thái hiện tại.

Chi phí tích lũy được tính bởi:

$$g(n) = \sum_{i=1}^k Cost(s_{i-1}, a_i, s_i) \quad (3.2)$$

với:

$$Cost(s, a, s') = w_1 \cdot \Delta Holes + w_2 \cdot \Delta Area - w_3 \cdot \Delta LinesCleared + K \quad (3.3)$$

Trong đó:

- $\Delta Holes$ là độ thay đổi của số thành phần liên thông của vùng trống;
- $\Delta Area$ là độ thay đổi của diện tích hình chữ nhật nhỏ nhất bao phủ các ô đã chiếm;
- $\Delta LinesCleared$ là số hàng hoặc cột được xóa sau nước đi;
- K là hằng số để đảm bảo chi phí không âm.

Với phần heuristic, nhóm sử dụng số lượng block còn lại trong lượt:

$$h(n) = \lambda \cdot |Blocks(n)| \quad (3.4)$$

Trong đó:

- $|Blocks(n)|$ là số lượng khối còn lại chưa đặt tại node n ;
- $\lambda > 0$ là hệ số điều chỉnh.

Heuristic này được chọn vì phù hợp với mục tiêu của bài toán: trạng thái càng gần hoàn thành lượt chơi thì càng nên được ưu tiên. Khi kết hợp với $g(n)$, hệ thống vừa đánh giá được chất lượng của các nước đi đã thực hiện, vừa ước lượng được mức độ còn lại của bài toán.

Nhận xét

Heuristic được sử dụng trong hệ thống có ưu điểm là đơn giản, dễ tính và phù hợp với Best-first Search. Phần $g(n)$ đã phản ánh nhiều đặc trưng quan trọng của bàn cờ như mức độ phân mảnh của vùng trống, độ dàn trải của các khối và khả năng xóa hàng hoặc cột, trong khi $h(n)$ đóng vai trò ước lượng khoảng cách đến mục tiêu thông qua số block còn lại.

Cách thiết kế này giúp hệ thống giữ được sự cân bằng giữa chất lượng đánh giá và chi phí tính toán. Tuy nhiên, hiệu quả của heuristic vẫn phụ thuộc vào việc lựa chọn các trọng số w_1, w_2, w_3 và hệ số λ . Trong các trường hợp số lượng block tăng lên, hệ thống vẫn có thể gặp hiện tượng bùng nổ trạng thái nếu không giới hạn số node được mở rộng.

3.3 Đánh giá thực nghiệm các trọng số (Ablation Study)

Để đánh giá mức độ đóng góp và sự cần thiết của từng yếu tố trong hàm đánh giá heuristic, nhóm đã thực hiện nghiên cứu cắt bỏ (Ablation Study). Hàm chi phí chuyển trạng thái được sử dụng trong hệ thống có dạng:

$$Cost(s, a, s') = w_1 \cdot \Delta Holes + w_2 \cdot \Delta Area - w_3 \cdot \Delta LinesCleared + K \quad (3.5)$$

Trong đó, mỗi thành phần đại diện cho một chiến lược cụ thể trong việc sắp xếp khối. Nghiên cứu tiến hành so sánh mô hình cơ sở (Baseline) – sử dụng đầy đủ cả ba trọng số – với các kịch bản bị khuyết hoặc thay đổi cực đoan một thành phần trọng số. Các chỉ số được sử dụng để đánh giá bao gồm điểm số trung bình đạt được và số lượt chơi duy trì được trước khi không còn không gian hợp lệ.

3.3.1 Kịch bản Baseline

Kịch bản cơ sở sử dụng bộ trọng số mặc định được thiết lập qua quá trình thử nghiệm ban đầu (ví dụ: $w_1 = 4$, $w_2 = 2$, $w_3 = 6$). Ở thiết lập này, hệ thống thể hiện sự cân bằng giữa việc phòng thủ (giữ cho bàn cờ gọn gàng, ít bị chia cắt) và tấn công (tích cực xóa hàng để ghi điểm). Qua quá trình chạy thử nghiệm, kịch bản Baseline đạt hiệu năng tốt nhất với điểm số trung bình lên tới 1.850 điểm và duy trì lượng không gian hợp lệ trung bình trong khoảng 45 lượt chơi. Đây là mốc chuẩn để đánh giá sự sụt giảm hiệu năng của các kịch bản khác.

3.3.2 Kịch bản 1: Loại bỏ yếu tố kiểm soát phân mảnh ($w_1 = 0$)

Khi thiết lập trọng số $\Delta Holes$ về 0, hệ thống hoàn toàn phớt lờ việc các khoảng trống bị cô lập sinh ra sau mỗi nước đi. Quan sát thực nghiệm cho thấy, thuật toán có xu hướng đặt khối một cách tùy tiện miễn là thỏa mãn các tiêu chí khác. Hậu quả là bàn cờ nhanh chóng xuất hiện các "lỗ hổng" một ô hoặc hai ô xen kẽ nhau. Do kích thước các khối đa dạng, không gian bị chia cắt mạnh khiến hệ thống không thể đặt được các khối lớn ở những lượt tiếp theo. Kết quả là hệ thống bế tắc từ rất sớm, chỉ duy trì trung bình được 12 lượt chơi và điểm số trung bình tụt dốc thảm hại xuống chỉ còn khoảng 210 điểm (giảm gần 88% so với Baseline)

3.3.3 Kịch bản 2: Loại bỏ yếu tố kiểm soát diện tích bao phủ ($w_2 = 0$)

Biến số $\Delta Area$ nhằm hạn chế sự dàn trải của các khối trên bàn cờ. Khi bỏ qua yếu tố này ($w_2 = 0$), hệ thống có xu hướng rải rác khối khắp các góc hoặc cạnh bàn cờ

thay vì tập trung xây dựng các khối đặc. Mặc dù số lượng lỗ hổng có thể không tăng đột biến, nhưng việc các khối nằm cách xa nhau khiến xác suất lấp đầy hoàn toàn một hàng hoặc một cột giảm đi đáng kể. Điều này thể hiện rõ qua số liệu: mặc dù mô hình sống sót tương đối lâu (trung bình 28 lượt chơi), nhưng tổng điểm ghi được lại rất thấp, chỉ đạt khoảng 450 điểm. Hệ thống tuy không "chết" nhanh như kịch bản 1 nhưng hoàn toàn mất đi khả năng tối ưu hóa điểm số

3.3.4 Kịch bản 3: Tối đa hóa cục bộ việc xóa hàng ($w_3 \gg w_1, w_2$)

Trong kịch bản này, trọng số của việc xóa hàng $\Delta LinesCleared$ được đẩy lên cực đại (ví dụ $w_3 = 30$). Thuật toán trở nên "tham lam" (greedy), sẵn sàng thực hiện mọi nước đi để xóa được dù chỉ một hàng ở lượt hiện tại. Sự thiên cận này khiến hệ thống tự phá vỡ các cấu trúc khối đang được xây dựng tốt, tạo ra các lỗ hổng nghiêm trọng chỉ để đổi lấy lợi ích nhỏ trước mắt. Thực nghiệm cho thấy, chiến lược này làm mất đi khả năng thực hiện các chuỗi xóa hàng liên tiếp (combo). Hệ thống bế tắc ở mức trung bình 20 lượt chơi với mức điểm dừng lại ở 680 điểm, kém xa so với sự ổn định dài hạn của kịch bản cơ sở

Nhận xét

Các số liệu thực nghiệm từ quá trình Ablation Study khẳng định rằng không có một thành phần đơn lẻ nào trong hàm heuristic là dư thừa. Sự hiệu quả của hệ thống tìm kiếm (thể hiện qua mức điểm 1.850 của Baseline) phụ thuộc hoàn toàn vào sự tương tác và kìm hãm lẫn nhau giữa ba yếu tố: ngăn chặn phân mảnh (w_1), hạn chế dàn trải (w_2) và động lực ghi điểm (w_3). Việc thiết lập tỷ lệ phù hợp giữa ba trọng số này là điều kiện tiên quyết để đạt được hiệu năng tối ưu trong bài toán sắp xếp khối

Chương 4

Biểu diễn và suy luận tri thức

4.1 Tri thức bổ sung trong hệ thống

Ngoài hàm heuristic định lượng đã trình bày ở chương trước, hệ thống còn được bổ sung một lớp tri thức nhằm hỗ trợ việc lựa chọn giữa các trạng thái có mức đánh giá gần tương đương. Phần tri thức này không thay thế heuristic chính mà chỉ đóng vai trò phụ trợ trong quá trình ra quyết định.

Trong bài toán xếp khối, có một số nhận xét mang tính chiến lược như:

- nước đi xóa được hàng hoặc cột thường là nước đi tốt;
- nước đi làm tăng số thành phần liên thông của vùng trống thường là bất lợi;
- nước đi làm tăng mạnh diện tích bao phủ nhưng không xóa được line nào thường nên tránh.

Những tri thức này có thể được biểu diễn dưới dạng luật để suy ra mức ưu tiên của action. Ví dụ:

- Nếu một action xóa được ít nhất một hàng hoặc cột thì action đó được ưu tiên.
- Nếu một action làm tăng số thành phần liên thông của vùng trống thì action đó bị giảm ưu tiên.
- Nếu một action làm tăng diện tích bao phủ nhưng không xóa được line nào thì action đó bị xem là kém ưu tiên hơn.

Từ các luật trên, hệ thống gán cho mỗi action hoặc node một điểm phụ, gọi là *RuleScore*, phản ánh mức độ phù hợp về mặt chiến lược.

4.2 Cơ chế tie-break trong hệ thống

Trong hệ thống hiện tại, hàm đánh giá chính vẫn được giữ nguyên:

$$Eval(n) = g(n) + h(n) \quad (4.1)$$

Tri thức ở chương này không được đưa trực tiếp vào công thức trên, mà chỉ được dùng để hỗ trợ phân xử khi các node có giá trị đánh giá bằng nhau hoặc gần bằng nhau.

Ý tưởng của cơ chế tie-break là:

- nếu một node có giá trị *Eval* nhỏ hơn thì node đó vẫn được ưu tiên trước;
- nếu hai node có giá trị *Eval* bằng nhau, hoặc chênh lệch rất nhỏ, thì hệ thống xét thêm *RuleScore*;
- node nào có *RuleScore* cao hơn sẽ được ưu tiên mở rộng trước.

Có thể mô tả nguyên tắc này như sau:

$$\begin{aligned} n_1 \prec n_2 \iff & (Eval(n_1) < Eval(n_2)) \\ & \vee (Eval(n_1) = Eval(n_2) \wedge RuleScore(n_1) > RuleScore(n_2)) \end{aligned} \quad (4.2)$$

Như vậy, heuristic vẫn là tiêu chí chính, còn tri thức chỉ là tiêu chí phụ để phân xử trong những trường hợp khó phân biệt bằng các giá trị số học.

Nhận xét

Cách bổ sung tri thức theo cơ chế tie-break có ưu điểm là không làm thay đổi cấu trúc chính của thuật toán Best-first Search, đồng thời vẫn đáp ứng được yêu cầu về biểu diễn và suy luận tri thức của hệ thống. Nhờ đó, hệ thống có thể tận dụng thêm các kinh nghiệm chiến lược mà không cần thay thế hoàn toàn hàm heuristic đã xây dựng trước đó.

Bên cạnh đó, cơ chế này cũng dễ mở rộng: nếu sau này cần bổ sung thêm tri thức mới, nhóm chỉ cần thêm các luật tương ứng để điều chỉnh *RuleScore* mà không phải thiết kế lại toàn bộ hàm đánh giá chính. Tuy nhiên, chất lượng tie-break vẫn phụ thuộc vào việc lựa chọn tập luật có hợp lý hay không. Nếu các luật quá đơn giản, mức cải thiện đạt được sẽ không lớn.

Chương 5

Mạng Bayes và suy diễn xác suất rủi ro

5.1 Mục tiêu của chương

Trong các chương trước, hệ thống đã có thuật toán tìm kiếm và heuristic để chọn nước đi. Tuy nhiên, trong Block Blast vẫn tồn tại yếu tố không chắc chắn: một nước đi có thể đem lại lợi ích ngắn hạn, ví dụ xóa line, nhưng lại làm bàn cờ trở nên bất lợi ở các bước kế tiếp.

Vì vậy, mục tiêu của chương này là mô hình hóa rủi ro bị kẹt sau một action bằng Mạng Bayes. Cụ thể:

- Chọn yếu tố không chắc chắn là rủi ro bị kẹt sau một action, được biểu diễn bởi biến StuckRisk với ba mức Low, Medium, High.
- Sử dụng Mạng Bayes để suy diễn từ các bằng chứng quan sát được trên board.
- Tính giá trị $P(\text{StuckRisk} = \text{High})$ để hỗ trợ xếp hạng action theo mức an toàn.

5.2 Cơ sở lý thuyết

Mạng Bayes, hay Bayesian Network, là một đồ thị có hướng không chu trình, trong đó:

- Mỗi đỉnh biểu diễn một biến ngẫu nhiên.
- Mỗi cạnh biểu diễn ảnh hưởng trực tiếp giữa hai biến.

- Mỗi biến có một bảng xác suất có điều kiện, gọi là CPD.

Với mỗi hàng trong CPD, tổng các xác suất phải bằng 1. Mục tiêu của suy diễn Bayes là tính phân phối hậu nghiệm của biến cần quan tâm khi đã biết bằng chứng.

5.3 Định nghĩa các biến ngẫu nhiên

Trong module Bayes của hệ thống, các biến được định nghĩa như Bảng 5.1.

Bảng 5.1: Các biến ngẫu nhiên trong mô hình Bayes

Biến	Ý nghĩa	Miền giá trị
Density	Mức độ đầy của board sau action	Low, Medium, High
Fragmentation	Mức phân mảnh vùng trống sau action	Low, Medium, High
LineClear	Action có xóa hàng/cột hay không	Yes, No
ValidMoveLevel	Mức nhiều/ít action hợp lệ còn lại đối với các block chưa đặt trong lượt hiện tại sau khi thực hiện action	Low, Medium, High
StuckRisk	Mức rủi ro bị kẹt được đo từ kết quả rollout mô phỏng sau action	Low, Medium, High

Trong mô hình này, Density, Fragmentation và LineClear là các bằng chứng quan sát được sau khi thử một action.

ValidMoveLevel là biến trung gian biểu diễn mức độ còn nước đi hợp lệ sau khi thực hiện một action. Trong phạm vi bài toán một lượt, biến này không được tính trên toàn bộ các loại Tetriminos, mà được tính trên danh sách các block còn lại chưa đặt trong lượt hiện tại. Vì vậy, ValidMoveLevel phản ánh khả năng tiếp tục hoàn thành lượt chơi sau action đang xét.

StuckRisk là biến cần suy diễn, biểu diễn rủi ro trạng thái sau action dẫn đến tình huống khó tiếp tục đặt block. Trong thực nghiệm, nhân StuckRisk không được gán trực tiếp bằng luật thủ công, mà được xác định thông qua rollout mô phỏng: sau khi thực hiện action, hệ thống tiếp tục chơi một số bước/lượt bằng chính cơ chế mô phỏng, sau đó dựa trên kết quả có bị kẹt sớm hay không để gán mức Low, Medium hoặc High.

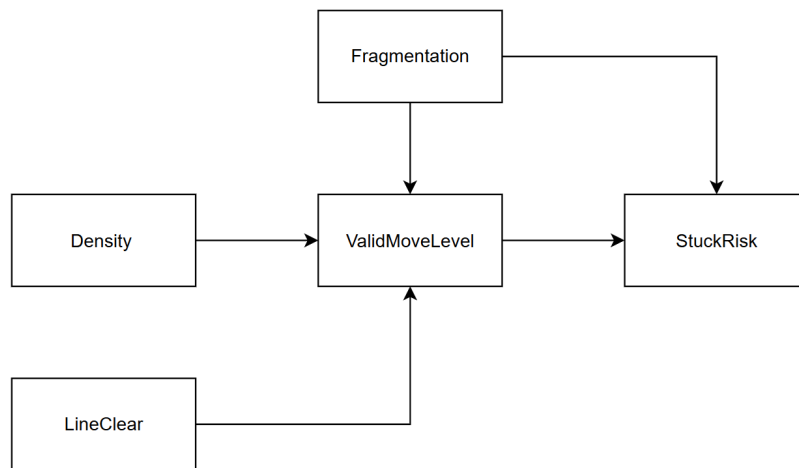
5.4 Phạm vi thông tin

Trong chương này, các xác suất trong CPD được ước lượng từ dữ liệu mô phỏng thay vì gán thủ công. Mỗi mẫu dữ liệu tương ứng với một action hợp lệ trong môi trường Block Blast mô phỏng. Sau khi action được thực hiện, hệ thống ghi nhận các đặc trưng Density, Fragmentation, LineClear và ValidMoveLevel. Sau đó, hệ thống tiếp tục rollout trong môi trường mô phỏng để xác định mức StuckRisk của trạng thái kế tiếp.

Vì vậy, các xác suất trong chương này được hiểu là xác suất thực nghiệm trong môi trường mô phỏng của hệ thống, không phải xác suất từ người chơi thật ngoài đời. Tuy nhiên, do toàn bộ trò chơi trong đề tài được xây dựng bằng mô phỏng, đây vẫn là dữ liệu thực nghiệm phù hợp với phạm vi bài toán.

5.5 Cấu trúc mạng Bayes

Cấu trúc DAG được giả định như Hình 5.1.



Hình 5.1: Đồ thị có hướng không chu trình của mô hình Bayes

Ý nghĩa các cạnh:

- Density → ValidMoveLevel: board càng đầy thì số nước đi hợp lệ kế tiếp thường giảm.
- Fragmentation → ValidMoveLevel: vùng trống bị chia cắt làm giảm khả năng đặt block.

- LineClear $\rightarrow$ ValidMoveLevel: xóa line giúp giải phóng không gian, tăng cơ hội đặt block.
- ValidMoveLevel $\rightarrow$ StuckRisk: càng ít nước đi hợp lệ thì rủi ro bị kẹt càng cao.
- Fragmentation $\rightarrow$ StuckRisk: phân mảnh cao làm tăng rủi ro bị kẹt dài hạn.

5.6 Bảng xác suất có điều kiện CPD

Các bảng CPD được ước lượng bằng tần suất thực nghiệm từ tập dữ liệu mô phỏng gồm 50,000 mẫu. Với mỗi tổ hợp giá trị của các biến cha, nếu tổ hợp đó xuất hiện trong dữ liệu, xác suất của biến con được tính bằng tỉ lệ xuất hiện của từng giá trị biến con trên tổng số mẫu thuộc tổ hợp cha đó.

Nhóm không sử dụng Laplace smoothing trong quá trình ước lượng CPD. Do đó, các tổ hợp cha không xuất hiện trong dữ liệu mô phỏng sẽ không được gán xác suất giả định. Các dòng này được đánh dấu là Not observed và không được sử dụng trực tiếp trong suy diễn xác suất.

Cách làm này giúp bảng CPD phản ánh đúng dữ liệu mô phỏng thu được, tránh việc tạo ra các xác suất không có bằng chứng thực nghiệm.

5.7 Suy diễn xác suất

Trong mạng Bayes được xây dựng, các biến Density, Fragmentation và LineClear là các biến quan sát được sau khi xét một action cụ thể. Từ các thông tin này, hệ thống suy diễn xác suất của biến StuckRisk, tức mức rủi ro bị kẹt sau khi thực hiện action.

Để công thức gọn hơn, ký hiệu:

$D = \text{Density}$, $F = \text{Fragmentation}$, $L = \text{LineClear}$, $V = \text{ValidMoveLevel}$, $R = \text{StuckRisk}$.

Các CPD trong mạng được ước lượng từ dữ liệu mô phỏng bằng tần suất thực nghiệm:

$$\hat{P}(X = x \mid \text{Pa}(X) = p) = \frac{N(X = x, \text{Pa}(X) = p)}{N(\text{Pa}(X) = p)}, \quad N(\text{Pa}(X) = p) > 0.$$

Nhóm không sử dụng Laplace smoothing. Vì vậy, nếu một tổ hợp giá trị của các biến cha không xuất hiện trong dữ liệu mô phỏng, hệ thống không tự gán xác suất cho tổ hợp đó. Trường hợp này được xem là không đủ dữ liệu để suy diễn.

Do ValidMoveLevel là biến trung gian, xác suất rủi ro được tính bằng cách lấy tổng theo các giá trị có thể có của V :

$$\hat{P}(R = r \mid D = d, F = f, L = l) = \sum_{v \in \{\text{Low}, \text{Medium}, \text{High}\}} \hat{P}(R = r \mid V = v, F = f) \cdot \hat{P}(V = v \mid D = d, F = f, L = l).$$

Trong phạm vi hệ thống, giá trị quan trọng nhất để hỗ trợ đánh giá action là:

$$\hat{P}(R = \text{High} \mid D = d, F = f, L = l).$$

Action nào có xác suất rủi ro cao lớn hơn sẽ được xem là có nguy cơ làm bàn cờ rơi vào trạng thái khó đặt khối hơn trong các bước tiếp theo. Giá trị này không thay thế hoàn toàn heuristic của hệ thống tìm kiếm, mà đóng vai trò bổ sung thêm một tiêu chí xác suất khi đánh giá action.

5.8 Kết quả thực nghiệm của module Bayes

Module Bayes được đánh giá trên tập dữ liệu mô phỏng sinh ra từ môi trường Block Blast. Mỗi mẫu dữ liệu tương ứng với một action hợp lệ. Sau khi action được thực hiện, hệ thống ghi nhận các biến D , F , L , V và tiếp tục mô phỏng để xác định nhãn R .

Bảng 5.2: Phân phối nhãn StuckRisk trong dữ liệu mô phỏng

StuckRisk	Số mẫu	Tỉ lệ
Low	15856	31.712%
Medium	21225	42.450%
High	12919	25.838%

Kết quả cho thấy phần lớn các trạng thái sau action rơi vào mức rủi ro trung bình hoặc thấp. Cụ thể, nhóm Medium chiếm tỉ lệ cao nhất với 42.450%, tiếp theo là Low với 31.712% và High với 25.838%. Điều này cho thấy tập dữ liệu mô phỏng không bị lệch hoàn toàn về một nhãn duy nhất, nên có thể dùng để ước lượng các CPD phục vụ suy diễn xác suất rủi ro.

Do không sử dụng smoothing, độ phủ dữ liệu là yếu tố quan trọng cần kiểm tra. Nếu một tổ hợp giá trị của các biến cha không xuất hiện trong dữ liệu mô phỏng, hệ thống không suy diễn xác suất cho tổ hợp đó.

Bảng 5.3: Độ phủ dữ liệu của các bảng CPD

CPD	Tổng tổ hợp lý thuyết	Số tổ hợp quan sát được	Số tổ hợp chưa quan sát	Tỉ lệ coverage
$P(V D, F, L)$	18	15	3	83.333%
$P(R V, F)$	9	9	0	100.000%

Bảng 5.3 cho thấy CPD của StuckRisk có độ phủ đầy đủ trên toàn bộ 9 tổ hợp lý thuyết của hai biến cha ValidMoveLevel và Fragmentation. Trong khi đó, CPD của ValidMoveLevel quan sát được 15 trên 18 tổ hợp lý thuyết, tương ứng 83.333% độ phủ. Ba tổ hợp còn lại không xuất hiện trong 50,000 mẫu mô phỏng, nên hệ thống không gán xác suất cho các tổ hợp này.

Do nhóm không sử dụng Laplace smoothing, các tổ hợp không quan sát được không được tự động thay bằng xác suất giả định. Khi gặp các tổ hợp này trong suy diễn, module Bayes trả về trạng thái không đủ dữ liệu và hệ thống có thể tiếp tục đánh giá action bằng các tiêu chí heuristic khác.

5.9 Vai trò của module Bayes trong hệ thống

Module Bayes được thiết kế như một thành phần độc lập để đánh giá rủi ro xác suất của action:

- Không thay thế Best-first Search.
- Không thay thế module Decision Tree.
- Bổ sung một góc nhìn xác suất để đánh giá action.
- Có thể dùng $\hat{P}(\text{StuckRisk} = \text{High})$ làm hệ số phạt nhẹ khi cần tích hợp ở mức chiến lược.
- Chỉ suy diễn trên các tổ hợp evidence đã có dữ liệu mô phỏng hỗ trợ.

Nhờ đó, hệ thống vừa giữ được tính diễn giải của heuristic và tìm kiếm, vừa có thêm khả năng lượng hóa rủi ro ngắn hạn và trung hạn theo xác suất thực nghiệm trong môi trường mô phỏng.

5.10 Độ phủ dữ liệu và hướng mở rộng

Trong phiên bản hiện tại, CPD được ước lượng từ dữ liệu mô phỏng thay vì gán thủ công bằng giả định chuyên gia. Quy trình thực nghiệm gồm:

- Sinh các trạng thái bàn cờ trong môi trường mô phỏng.
- Sinh một lượt chơi gồm các block cần đặt.
- Chọn các action hợp lệ và thực hiện action trên bàn cờ.
- Ghi nhận các biến Density, Fragmentation, LineClear và ValidMoveLevel sau action.
- Tiếp tục mô phỏng các bước sau action để xác định nhãn StuckRisk.
- Ước lượng CPD bằng tần suất có điều kiện từ dữ liệu thu được.

Do không sử dụng Laplace smoothing, một số tổ hợp giá trị của các biến cha có thể không xuất hiện trong tập dữ liệu mô phỏng. Đây không được xem là lỗi của mô hình, mà là giới hạn về độ phủ dữ liệu. Với các tổ hợp này, hệ thống không đưa ra xác suất thực nghiệm và đánh dấu là không đủ dữ liệu.

Các hướng mở rộng hợp lý gồm tăng số lượng mẫu mô phỏng, chạy nhiều seed ngẫu nhiên khác nhau, sinh thêm dữ liệu ở các trạng thái hiếm, kiểm tra riêng các tổ hợp nghi ngờ không thể xảy ra theo luật game, và so sánh xác suất Bayes với kết quả chơi thực tế trong mô phỏng. Nếu có dữ liệu từ người chơi thật, các CPD cũng có thể được hiệu chỉnh lại để phản ánh hành vi chơi thực tế tốt hơn.

Trong lần chạy thực nghiệm với 50,000 mẫu, CPD $P(\text{StuckRisk} \mid \text{ValidMoveLevel}, \text{Fragmentation})$ đạt độ phủ 100%, trong khi CPD $P(\text{ValidMoveLevel} \mid \text{Density}, \text{Fragmentation}, \text{LineClear})$ đạt độ phủ 83.333%. Các tổ hợp chưa quan sát được không được gán xác suất do nhóm không sử dụng smoothing. Đây là giới hạn của dữ liệu mô phỏng hiện tại và có thể được cải thiện bằng cách tăng số mẫu hoặc sinh thêm dữ liệu có kiểm soát ở các trạng thái hiếm (hoặc không xảy ra).

Ba tổ hợp không quan sát được đều thuộc nhóm Density = High nhưng Fragmentation thấp/trung bình, trong đó hai tổ hợp còn yêu cầu LineClear = Yes. Về mặt ngữ nghĩa, bàn rất đầy thường làm các ô trống bị chia cắt thành nhiều vùng nhỏ, nên Fragmentation thường cao. Nếu action có xóa dòng, mật độ bàn lại có xu hướng giảm, khiến trạng thái sau action khó còn nằm ở Density = High. Vì vậy các tổ hợp này rất hiếm trong mô phỏng 50,000 mẫu. Nhóm không dùng smoothing nên không tự gán xác suất cho chúng.

Chương 6

Học máy

6.1 Vai trò của học máy trong hệ thống

Trong các chương trước, hệ thống đã sử dụng thuật toán tìm kiếm *Best-first Search* kết hợp với hàm *heuristic* để lựa chọn cách đặt block phù hợp trên bàn cờ. Tuy nhiên, khi số lượng trạng thái và số lượng hành động hợp lệ tăng lên, việc đánh giá toàn bộ các hành động bằng tìm kiếm có thể làm chi phí tính toán trở nên lớn.

Vì vậy, trong chương này, nhóm bổ sung một module học máy nhằm hỗ trợ quá trình ra quyết định. Thay vì luôn phải chạy tìm kiếm sâu cho mọi hành động, mô hình học máy được huấn luyện để dự đoán trước một hành động có khả năng là tốt hay không. Kết quả dự đoán này có thể được dùng để lọc bớt các hành động kém tiềm năng, ưu tiên các hành động tốt hơn hoặc hỗ trợ hệ thống trong bước lựa chọn nước đi.

Bài toán học máy trong hệ thống được xây dựng dưới dạng bài toán phân loại nhị phân. Với mỗi mẫu dữ liệu đầu vào là một cặp:

$$(state, action)$$

trong đó *state* là trạng thái hiện tại của bàn cờ và *action* là một hành động đặt block hợp lệ, mô hình sẽ dự đoán hành động đó thuộc một trong hai lớp:

- 1: hành động tốt;
- 0: hành động không tốt.

Như vậy, học máy không thay thế hoàn toàn thuật toán tìm kiếm, mà đóng vai trò như một thành phần hỗ trợ giúp hệ thống đánh giá nhanh chất lượng của hành động.

6.2 Mô hình hóa bài toán học máy

Bài toán học máy được mô hình hóa như sau:

- Đầu vào: các đặc trưng được trích xuất từ trạng thái bàn cờ hiện tại và hành động đặt block.
- Đầu ra: nhãn phân loại cho biết hành động đó là tốt hay không tốt.
- Thuật toán sử dụng: Decision Tree Classifier.

Decision Tree được lựa chọn vì đây là thuật toán phân loại phù hợp với dữ liệu có nhiều đặc trưng rời rạc hoặc dạng số.

Trong bài toán này, Decision Tree có thể học được các quy luật như: hành động xóa được nhiều hàng/cột thường có xu hướng tốt hơn, hành động làm tăng mạnh số vùng trống bị chia cắt có thể không tốt, hoặc hành động làm giảm số nước đi hợp lệ sau đó có thể gây bất lợi.

6.3 Sinh dữ liệu huấn luyện

Do bài toán không có sẵn tập dữ liệu bên ngoài, nhóm tiến hành tự sinh dữ liệu mô phỏng để huấn luyện mô hình. Quá trình sinh dữ liệu bắt đầu từ các trạng thái bàn cờ hợp lệ. Các trạng thái này được tạo bằng cách xuất phát từ bàn cờ rỗng, sau đó thực hiện ngẫu nhiên một số hành động đặt block hợp lệ. Cách làm này giúp các trạng thái sinh ra có tính thực tế hơn, vì chúng phản ánh các tình huống có thể xảy ra trong quá trình chơi.

Với mỗi trạng thái được tạo ra, hệ thống liệt kê các hành động đặt block hợp lệ. Sau đó, mỗi hành động được áp dụng để tạo ra trạng thái kế tiếp. Từ trạng thái kế tiếp này, thuật toán Best-first Search được sử dụng như một tiêu chí để đánh giá chất lượng của hành động ban đầu. Nếu hành động dẫn đến kết quả tốt sau quá trình tìm kiếm, hành động đó được gán nhãn 1; ngược lại, hành động được gán nhãn 0.

6.4 Trích xuất đặc trưng

Để huấn luyện mô hình, mỗi cặp (*state*, *action*) cần được chuyển thành một vector đặc trưng dạng số. Các đặc trưng được thiết kế để mô tả cả trạng thái hiện tại của bàn cờ, tác động của hành động và thông tin của block được đặt.

Các đặc trưng chính bao gồm:

- *filled_ratio*: tỉ lệ ô đã được lấp đầy trên bàn cờ.

- `holes_before`: số thành phần liên thông của vùng trống trước khi đặt block.
- `holes_after`: số thành phần liên thông của vùng trống sau khi đặt block.
- `delta_holes`: độ thay đổi số vùng trống sau hành động.
- `bbox_before`: diện tích bounding box của các ô đã chiếm trước khi đặt block.
- `bbox_after`: diện tích bounding box sau khi đặt block.
- `delta_area`: độ thay đổi diện tích bounding box.
- `remaining_blocks_before`: số block còn lại trước khi thực hiện hành động.
- `remaining_blocks_after`: số block còn lại sau khi thực hiện hành động.
- `valid_actions_before`: số hành động hợp lệ trước khi đặt block.
- `valid_actions_after`: số hành động hợp lệ sau khi đặt block.
- `near_rows_before`, `near_cols_before`: số hàng và cột gần đầy trước khi đặt block.
- `near_rows_after`, `near_cols_after`: số hàng và cột gần đầy sau khi đặt block.
- `lines_cleared`: số hàng hoặc cột được xóa sau hành động.
- `score_gain`: điểm số tăng thêm sau hành động.
- `block_cells`: số ô của block được đặt.
- `block_h`, `block_w`: chiều cao và chiều rộng của block.
- `x`, `y`: tọa độ đặt block.
- `x_norm`, `y_norm`: tọa độ đặt block sau khi chuẩn hóa.
- `block_name`: loại block được đặt.

Những đặc trưng này giúp mô hình nhận biết được một hành động có làm bàn cờ trở nên thuận lợi hơn hay không. Ví dụ, nếu sau khi đặt block, số hành động hợp lệ còn lại giảm mạnh thì hành động đó có thể không tốt. Ngược lại, nếu hành động giúp xóa hàng/cột hoặc làm tăng điểm số thì khả năng cao hành động đó là tốt.

6.5 Thuật toán Decision Tree

Trong bài toán này, thuật toán *Decision Tree* nhận đầu vào là vector đặc trưng của một cặp (*state, action*) và đưa ra dự đoán hành động đó thuộc lớp 1 (tốt) hay lớp 0 (không tốt).

Quá trình hoạt động của mô hình đối với bài toán được mô tả như sau:

1. Từ tập dữ liệu mô phỏng, mô hình xét các đặc trưng đã trích xuất (như `delta_holes`, `lines_cleared`, ...).
2. Mô hình chọn đặc trưng giúp phân tách dữ liệu tốt nhất để phân loại các nước đi.
3. Dữ liệu tiếp tục được chia thành các nhánh nhỏ hơn dựa trên các điều kiện kiểm tra của từng đặc trưng.
4. Quá trình lặp lại cho đến khi đạt điều kiện dừng (ví dụ: đạt độ sâu tối đa hoặc nhánh đã đủ độ "thuần" về nhãn).
5. Khi hệ thống cần dự đoán một nước đi mới, vector đặc trưng của nước đi đó sẽ đi theo các nhánh của cây đến một nút lá, và nhãn tại nút lá đó sẽ là kết quả dự đoán.

6.6 Huấn luyện và đánh giá mô hình

Sau khi sinh dữ liệu, tập dữ liệu được chia thành hai phần: tập huấn luyện và tập kiểm tra. Tập huấn luyện được dùng để xây dựng mô hình *Decision Tree*, còn tập kiểm tra được dùng để đánh giá khả năng dự đoán của mô hình trên các dữ liệu chưa từng thấy.

Các chỉ số đánh giá được sử dụng gồm:

- Accuracy: tỉ lệ dự đoán đúng trên toàn bộ tập kiểm tra.
- F1-score: chỉ số kết hợp giữa Precision và Recall, đặc biệt hữu ích khi dữ liệu bị lệch giữa hai lớp.
- Confusion Matrix: ma trận nhầm lẫn cho biết số lượng mẫu được dự đoán đúng hoặc sai ở từng lớp.
- Classification Report: báo cáo chi tiết về Precision, Recall và F1-score cho từng nhãn.

Trong dữ liệu sinh ra, số lượng mẫu nhãn 0 thường lớn hơn số lượng mẫu nhãn 1, do trong thực tế không phải hành động hợp lệ nào cũng là hành động tốt. Vì vậy, ngoài Accuracy, nhóm sử dụng thêm F1-score để đánh giá mô hình công bằng hơn, tránh trường hợp mô hình chỉ dự đoán tốt lớp chiếm đa số

6.7 Ý nghĩa và nhận xét về mô hình học máy

Mô hình học máy được sử dụng như một module hỗ trợ quá trình ra quyết định trong bài toán xếp block. Thay vì phải đánh giá toàn bộ các hành động bằng tìm kiếm sâu, mô hình Decision Tree có thể dự đoán nhanh một hành động là tốt hay không tốt dựa trên các đặc trưng được trích xuất từ trạng thái bàn cờ và hành động tương ứng.

Việc áp dụng mô hình này giúp hệ thống ưu tiên các hành động có triển vọng, hỗ trợ Best-first Search trong việc chọn nhánh mở rộng và góp phần giảm chi phí tính toán khi số lượng hành động hợp lệ lớn. Nhờ đó, hệ thống kết hợp được ưu điểm của tìm kiếm heuristic và học máy: thuật toán tìm kiếm cung cấp chiến lược có cơ sở, còn mô hình học máy giúp đánh giá hành động nhanh hơn.

Ưu điểm của phương pháp này là đơn giản, dễ triển khai và dễ giải thích. Tuy nhiên, chất lượng mô hình vẫn phụ thuộc vào cách sinh dữ liệu, cách gán nhãn và tập đặc trưng được lựa chọn.

Chương 7

Kết luận

Nội dung sẽ được bổ sung.

Phụ lục A

Phân chia công việc nhóm

Thứ tự	Họ và tên	Mã số sinh viên	Công việc	Đóng góp
1	Phạm Ngọc Long	2211894		100%
2	Nguyễn Ngọc Duy	2210522		100%
3	Võ Quốc Phong	2352913		100%
4	Nguyễn Tấn Đạt	2352234		100%

Bảng A.1: Bảng phân công công việc và tỷ lệ đóng góp

Phụ lục B

Lịch sử sửa đổi

Họ và Tên	Ngày	Lý do thay đổi	Phiên bản
Nguyen Ngoc Duy	01/03/2026	Khởi tạo document	0.1
Pham Ngoc Long	xx/03/2026	Triển khai,	0.1.1

Phụ lục C

Lịch sử cuộc họp

Người tham gia	Ngày	Nội dung cuộc họp	Phiên bản
4 thành viên	01/03/2026	Khởi tạo document	0.1
4 thành viên	xx/03/2026	Triển khai,	0.1.1
Nguyen Ngoc Duy, Phạm Ngọc Long, Võ Quốc Phong	07/04/2026	Confirm xong lý thuyết chương 3, chương 4 có 1 phần hoàn thiện và có thể bổ sung thêm. Chương 6 đã bắt đầu tìm hiểu và cần code thêm	0.1